SIMATS ENGINEERING

ASSESSMENT TOOL 2

APPLICATION BASED QUESTIONS

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1107

Course Outcome Covered: CO2: Design UML diagrams to represent system requirements and architecture. (BL3)

Assessment Tool Weightage: 60%

Questions: 6 **Total Marks:** 60

Problems Solved

Problem No.	UML Diagram	System Modelled
1	Use Case Diagram	Smart Pharmacy Management System
2	Class Diagram	Smart Library Management System
3	Sequence Diagram	Online Examination System
4	Activity Diagram	Loan Approval System
5	Component Diagram	Smart E-Commerce Platform
6	Deployment Diagram	Smart Agriculture Monitoring System

Student Details

Particular	Details
Name of the Student	M Nishanth
Register Number	192511154
Department / Year	B. Tech CSE / 2nd Year
Date of Submission	20-10-2026

Annexure A: Code of Conduct

APPLICATION BASED QUESTIONS

I ____**M Nishanth (192511154)** ____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____**M Nishanth**_____

Date: _____**20-10-2026**_____

1. Introduction and Approach

Each problem in this assessment describes a system whose design is incomplete or unclear. The task is to analyse the system, find what is missing, and express the improved design as a UML diagram. Different diagrams show different views of a system: use case diagrams show requirements, class and component diagrams show structure, sequence and activity diagrams show behaviour, and deployment diagrams show the physical architecture (Booch, Rumbaugh, & Jacobson, 2005; Object Management Group, 2017). Together they meet Course Outcome CO2.

1.1 Problems Solved

Problem	UML Diagram	View of the System	Marks
1	Use Case Diagram	Requirements	10
2	Class Diagram	Static structure	10
3	Sequence Diagram	Interaction over time	10
4	Activity Diagram	Business workflow	10
5	Component Diagram	Software architecture	10
6	Deployment Diagram	Physical architecture	10
	Total		**60**

1.2 Method Followed for Every Problem

- **Understanding of the problem context:** the scenario is analysed, the stated requirements are listed, and the gaps or weaknesses in the current design are identified.
- **Application of concepts:** the UML diagram type is justified and the notation that will be used is explained.
- **Problem-solving approach:** the analysis is carried out step by step in tables (elements, relationships, flows) before the diagram is drawn.
- **Accuracy of solution:** the finished diagram is checked against the requirements and the UML notation rules.
- **Clarity:** an interpretation explains how the diagram improves the system, with its assumptions and limits.

1.3 Layout of this Document

Each complete diagram is placed on its own landscape page so that it can be read clearly, and the supporting tables are on the portrait pages before and after it. All diagrams follow UML 2.5.1 notation.

2. Problem 1: Use Case Diagram (Smart Pharmacy Management System)

2.1 Understanding of the Problem Context

A Smart Pharmacy Management System lets customers search medicines, upload prescriptions, place orders and pay online. Pharmacists verify prescriptions and dispense medicines, and administrators manage inventory and user accounts. The problem states that actor responsibilities and interactions are not clearly defined, so the analysis must find the gaps and add the missing behaviour.

Stated requirements:

- R1: Customers search medicines. R2: Customers upload prescriptions.
- R3: Customers place medicine orders. R4: Customers make online payments.
- R5: Pharmacists verify prescriptions. R6: Pharmacists dispense medicines.
- R7: Administrators manage inventory. R8: Administrators manage user accounts.

Gaps found in the current design:

- G1: There is no login step to protect customer orders and staff functions.
- G2: There is no check that a medicine is in stock before an order is accepted.
- G3: An invalid prescription has no path; it can only be accepted.
- G4: Stock levels are not updated after dispensing or inventory changes.
- G5: Customers cannot follow their order or be told about its progress.
- G6: The external payment gateway is not identified as an actor.

2.2 Application of Concepts

A **use case diagram** is applied because the problem is about requirements and actor responsibilities. It shows who uses the system and what each actor can do, without describing how it is built (Jacobson, Christerson, Jonsson, & Övergaard, 1992; Cockburn, 2001). The three relationships are used for three purposes: **«include»** for behaviour that is always needed and reused, **«extend»** for optional or conditional behaviour, and **generalization** for a specialised form of a general use case. Actors are drawn as stick figures outside the system boundary and use cases as ovals inside it.

2.3 Problem-Solving Approach

Step 1: Identify all actors.

Actor	Type	Responsibility
Customer	Primary	Searches medicines, uploads prescriptions, places and tracks orders, and pays online.
Pharmacist	Primary (staff)	Verifies prescriptions and dispenses medicines.
Administrator	Primary (staff)	Manages inventory and user accounts.
Payment Gateway	Secondary (external system)	Authorizes online payments on behalf of the system.

Step 2: List the given and the missing use cases.

ID	Use Case	Actor	Source	Description
UC1	Search Medicines	Customer	Given	Look up medicines by name or use.
UC2	Upload Prescription	Customer	Given	Submit a prescription for medicines that need one.
UC3	Place Order	Customer	Given	Order the selected medicines.
UC4	Make Online Payment	Customer, Gateway	Given	Pay for the order online.
UC5	Verify Prescription	Pharmacist	Given	Check that the prescription is valid.
UC6	Dispense Medicine	Pharmacist	Given	Hand over or send the medicines.
UC7	Manage Inventory	Administrator	Given	Add, update or remove stock items.
UC8	Manage User Accounts	Administrator	Given	Create, block or update accounts.
UC9	Authenticate User	Customer	Added (G1)	Log in before using protected functions.
UC10	Check Medicine Availability	Customer	Added (G2)	Confirm stock before accepting an order.

ID	Use Case	Actor	Source	Description
UC11	Reject Prescription	Pharmacist	Added (G3)	Reject an invalid prescription.
UC12	Update Stock Levels	Pharmacist, Administrator	Added (G4)	Adjust stock after dispensing or changes.
UC13	Track Order	Customer	Added (G5)	View the status of an order.
UC14	Send Notification	Customer	Added (G5)	Tell the customer about status changes.
UC15	Pay by Card	Customer, Gateway	Added	Payment by debit or credit card.
UC16	Pay by UPI	Customer, Gateway	Added	Payment by UPI.

Step 3: Define the relationships.

From	Relationship	To	Justification
Place Order	«include»	Check Medicine Availability	Stock must always be checked before an order is accepted.
Place Order	«include»	Make Online Payment	Every order must be paid.
Place Order	«include»	Authenticate User	Orders need a logged-in customer.
Track Order	«include»	Authenticate User	Order data is private to the customer.
Upload Prescription	«extend»	Place Order	Needed only when the medicine requires a prescription.
Dispense Medicine	«include»	Verify Prescription	A prescription must be verified before dispensing.
Dispense Medicine	«include»	Update Stock Levels	Stock falls every time medicine is dispensed.
Manage Inventory	«include»	Update Stock Levels	Inventory changes use the same stock update.
Reject Prescription	«extend»	Verify Prescription	Happens only when the prescription is invalid.

From	Relationship	To	Justification
Send Notification	«extend»	Track Order	Sent only when the order status changes.
Pay by Card	Generalization	Make Online Payment	Specialised payment method.
Pay by UPI	Generalization	Make Online Payment	Specialised payment method.

2.4 Optimized Use Case Diagram

The optimized Use Case Diagram is given as Figure 1 on the next page.

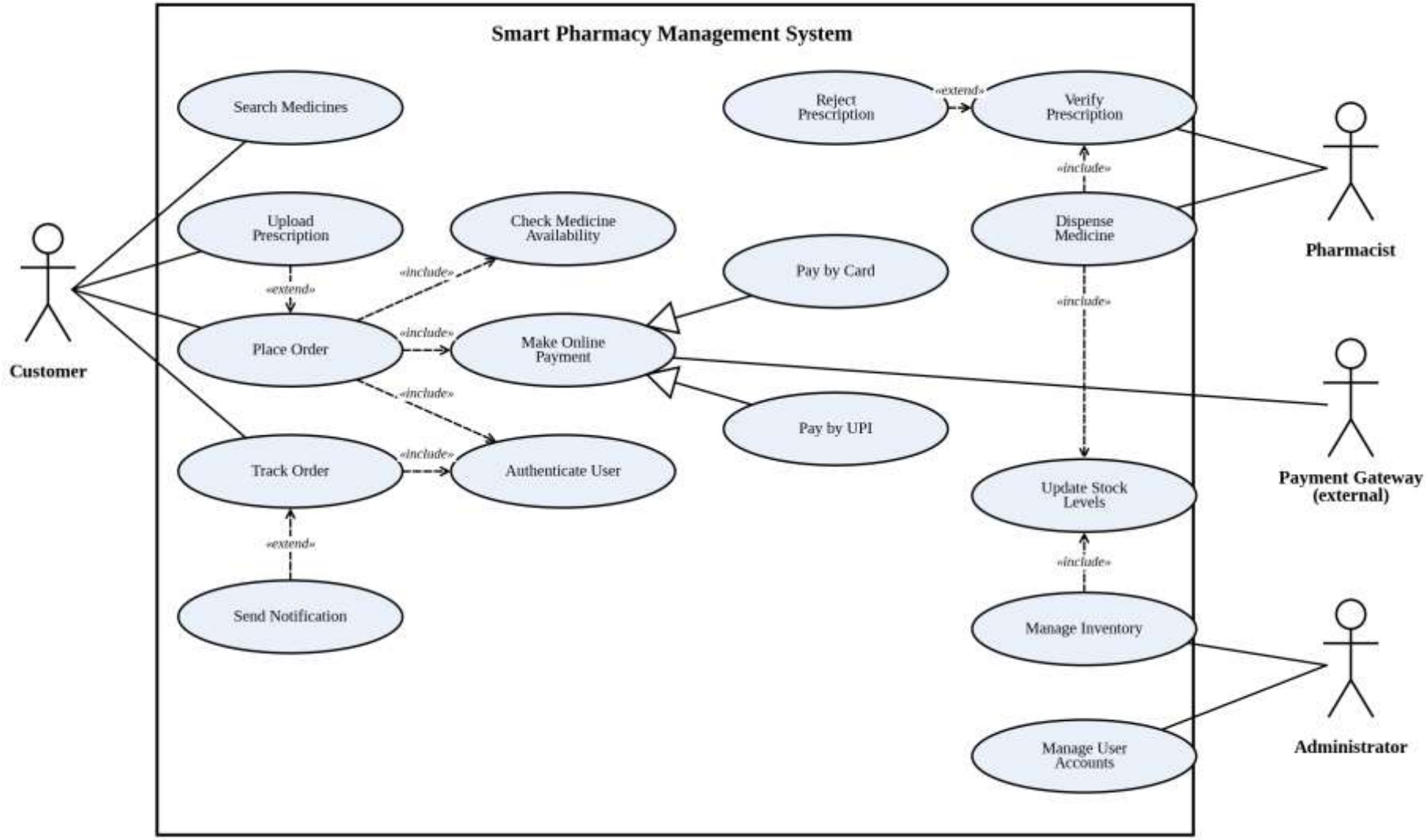


Figure 1: Use Case Diagram of the Smart Pharmacy Management System

2.5 Accuracy of the Solution

Check	Result
All eight stated requirements (R1 to R8) are covered by a use case.	Satisfied
All six gaps (G1 to G6) are closed by an added use case or actor.	Satisfied
Every actor is linked to at least one use case, and the external Payment Gateway is drawn outside the boundary.	Satisfied
«include» arrows run from the base to the included use case; «extend» arrows run from the extension to the base.	Satisfied
Generalization arrows (hollow triangle) point to the general use case, Make Online Payment.	Satisfied
Update Stock Levels is shared by two use cases, so it is defined once.	Satisfied

2.6 Interpretation and Clarity of the Solution

- **Responsibilities are now clear.** The customer orders and pays, the pharmacist verifies and dispenses, the administrator manages data, and the gateway handles payments only.
- **Reuse through «include».** Authenticate User and Update Stock Levels are each written once and used by several use cases, so a change is made in one place.
- **Conditional behaviour through «extend».** Only prescription medicines need an upload, and only invalid prescriptions are rejected, so the normal flow stays simple and safe.
- **Generalization allows growth.** New payment methods can be added under Make Online Payment without changing the order process.
- **Assumptions.** The added use cases are inferred from the gaps and should be confirmed with the pharmacy owner.

3. Problem 2: Class Diagram (Smart Library Management System)

3.1 Understanding of the Problem Context

The Smart Library Management System has six named entities: Book, Member, Librarian, Reservation, Fine and BorrowRecord. The problem states that their attributes, operations and relationships are not specified. They were worked out from the following business rules.

- BR1: A member can borrow many books over time. Each borrowing is recorded in a BorrowRecord for one member and one book.
- BR2: A librarian issues books, so a librarian is linked to many borrow records.
- BR3: A fine arises only from a late return of a borrow record and cannot exist without it.
- BR4: A member can reserve a book, and a book can have many reservations.
- BR5: Members and librarians are both people with common details and a login.
- BR6: A library holds many books, and a book can exist outside one library's record, for example when transferred.

Two classes were added to the six named entities: **Person** (to remove repeated data, rule BR5) and **Library** (to show aggregation, rule BR6).

3.2 Application of Concepts

A **class diagram** is applied because the problem is about the structure of the system (Fowler, 2003). Four relationship types are used: **association** (a plain link with a name and multiplicity), **aggregation** (hollow diamond at the whole, parts can exist alone), **composition** (filled diamond at the whole, parts are deleted with it) and **inheritance** (hollow triangle at the parent class). Attributes are private (-) and methods are public (+) to support encapsulation.

3.3 Problem-Solving Approach

Step 1: Identify classes, attributes and methods.

Class	Attributes	Methods
Person (abstract)	- personId: String - name: String - email: String - phone: String	+ login(): boolean + logout(): void

Class	Attributes	Methods
Member	- membershipNo: String - joinDate: Date - maxBooks: int	+ searchBook(): List + borrowBook(): BorrowRecord + reserveBook(): Reservation + payFine(): void
Librarian	- employeeId: String - designation: String	+ issueBook(): void + returnBook(): void + collectFine(): void
Book	- isbn: String - title: String - author: String - status: String	+ isAvailable(): boolean + updateStatus(): void
BorrowRecord	- recordId: String - issueDate: Date - dueDate: Date - returnDate: Date	+ isOverdue(): boolean + calculateFine(): double
Reservation	- reservationId: String - reservationDate: Date - status: String	+ confirm(): void + cancel(): void
Fine	- fineId: String - amount: double - paid: boolean	+ calculateAmount(): double + markPaid(): void
Library	- name: String - address: String	+ addBook(book): void + removeBook(book): void

Step 2: Define the relationships.

From	Relationship	To	Meaning
Member	Inheritance	Person	A member is a kind of person.
Librarian	Inheritance	Person	A librarian is a kind of person.
Member	Association (borrows)	BorrowRecord	Each borrowing belongs to one member.
Librarian	Association (issues)	BorrowRecord	A librarian issues books.

From	Relationship	To	Meaning
Book	Association (recorded in)	BorrowRecord	Each borrowing is of one book.
BorrowRecord	Composition (incurs)	Fine	A fine is deleted with its borrow record.
Member	Association (makes)	Reservation	A member reserves books.
Book	Association (reserved by)	Reservation	A book can be reserved many times.
Library	Aggregation (holds)	Book	A library holds books that can exist on their own.

Step 3: Specify the multiplicity.

Association	First End	Second End	Justification
Member – BorrowRecord	1	0..*	A member may have no borrowing or many.
Librarian – BorrowRecord	1	0..*	One librarian issues many books.
Book – BorrowRecord	1	0..*	A book is borrowed many times over time.
BorrowRecord – Fine	1	0..1	A fine exists only if the book was returned late.
Member – Reservation	1	0..*	A member may make many reservations.
Book – Reservation	1	0..*	A book can be reserved by many members, one at a time.
Library – Book	1	1..*	A library holds at least one book.

3.4 Well-Structured Class Diagram

The Class Diagram is given as Figure 2 on the next page.

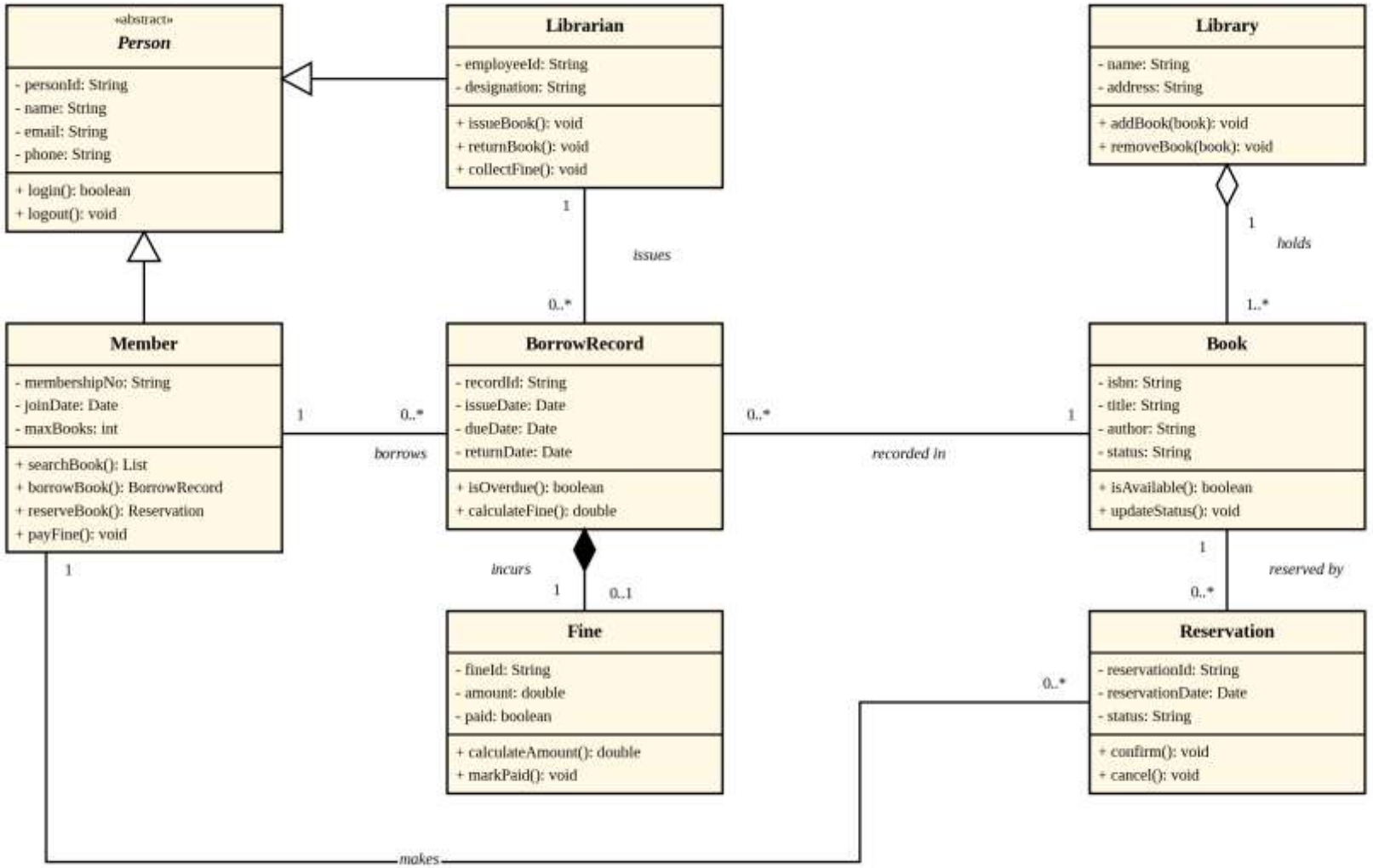


Figure 2: Class Diagram of the Smart Library Management System

3.5 Accuracy of the Solution

Check	Result
All six named classes (Book, Member, Librarian, Reservation, Fine, BorrowRecord) are present.	Satisfied
All four relationship types (association, aggregation, composition, inheritance) are used where appropriate.	Satisfied
Every association has a name and a multiplicity at both ends.	Satisfied
The composition diamond is filled and sits at the whole (BorrowRecord); the aggregation diamond is hollow and sits at Library.	Satisfied
The inheritance triangle points to the abstract parent Person.	Satisfied

3.6 Interpretation and Clarity of the Solution

- **BorrowRecord is the transaction class.** It links a member, a book and the librarian who issued it, and holds the dates needed to decide whether the book is overdue.
- **Composition for Fine.** A fine has no meaning without its borrow record, so it is deleted with it. A book is not owned by a library record in the same way, so Library–Book is aggregation.
- **Inheritance removes repetition.** Login and personal details are written once in Person and reused by Member and Librarian.
- **Limitations.** Loan period rules, fine rates and member categories (for example student and staff) are not modelled and can be added as attributes or subclasses.

4. Problem 3: Sequence Diagram (Online Examination System)

4.1 Understanding of the Problem Context

An Online Examination System lets students log in, select an examination, answer questions, submit responses and receive results. The participants named in the problem are the student, the examination module, the evaluation module and the database. The problem states that the communication between them is incomplete.

The interaction was analysed step by step, and the following **missing message exchanges** were found and added:

- M1: Login is never checked against stored users, so a login validation exchange with the database is added.
- M2: The exam and its questions are never loaded from the database before they are shown.
- M3: Answers are not saved as the student works, so work could be lost; each answer is saved to the database.
- M4: The final responses are not stored at submission.
- M5: The evaluation module does not fetch the answer key it needs to mark the responses.
- M6: The calculated score is not saved.
- M7: The result does not travel back to the student through the examination module.

A **LoginModule** object was added to the four named participants to handle M1.

4.2 Application of Concepts

A **sequence diagram** is applied because the question is about the order of messages between objects over time (Fowler, 2003). Each object has a **lifeline** (dashed vertical line), each period of work is shown as an **activation bar**, calls are solid arrows with filled heads, returns are dashed arrows, and a **loop fragment** is used for repeated answering.

4.3 Problem-Solving Approach

Step 1: Identify the participating objects.

Object	Role	Responsibility
Student	Actor	Logs in, selects the exam, answers and submits.

Object	Role	Responsibility
:LoginModule	Added	Checks the student's credentials.
:ExaminationModule	Named	Delivers questions, saves answers and collects the submission.
:EvaluationModule	Named	Marks the responses and calculates the score.
:Database	Named	Stores users, exams, answers, answer keys and results.

Step 2: Define the complete message flow.

No.	From	To	Message	Type
1	Student	LoginModule	login(id, password)	Synchronous
2	LoginModule	Database	validateUser(id, password)	Synchronous
3	Database	LoginModule	userRecord	Return
4	LoginModule	Student	loginSuccess	Return
5	Student	ExaminationModule	selectExam(examId)	Synchronous
6	ExaminationModule	Database	getExam(examId)	Synchronous
7	Database	ExaminationModule	examWithQuestions	Return
8	ExaminationModule	Student	displayQuestions()	Return
9	Student	ExaminationModule	answerQuestion(qId, answer)	Synchronous (loop)
10	ExaminationModule	Database	saveAnswer(qId, answer)	Synchronous (loop)
11	Database	ExaminationModule	answerSaved	Return (loop)
12	Student	ExaminationModule	submitExam(examId)	Synchronous
13	ExaminationModule	Database	storeResponses(examId)	Synchronous
14	Database	ExaminationModule	submissionStored	Return
15	ExaminationModule	EvaluationModule	evaluate(examId)	Synchronous
16	EvaluationModule	Database	getAnswerKey(examId)	Synchronous
17	Database	EvaluationModule	answerKey	Return
18	EvaluationModule	EvaluationModule	calculateScore()	Self

No.	From	To	Message	Type
19	EvaluationModule	Database	saveResult(studentId, score)	Synchronous
20	Database	EvaluationModule	resultSaved	Return
21	EvaluationModule	ExaminationModule	score	Return
22	ExaminationModule	Student	displayResult(score)	Return

Step 3: Define the lifelines and activation bars.

Object	Active During Messages	Reason
Student	Lifeline only	The student is an actor outside the system.
:LoginModule	1–4	Active from receiving the login until it replies.
:ExaminationModule	5–8, 9–11 and 12–22	Active while serving each request of the student.
:EvaluationModule	15–21	Active from the evaluate request until the score is returned.
:Database	2–3, 6–7, 10–11, 13–14, 16–17, 19–20	Active briefly for each read or write.

4.4 Detailed Sequence Diagram

The detailed Sequence Diagram is given as Figure 3 on the next page.

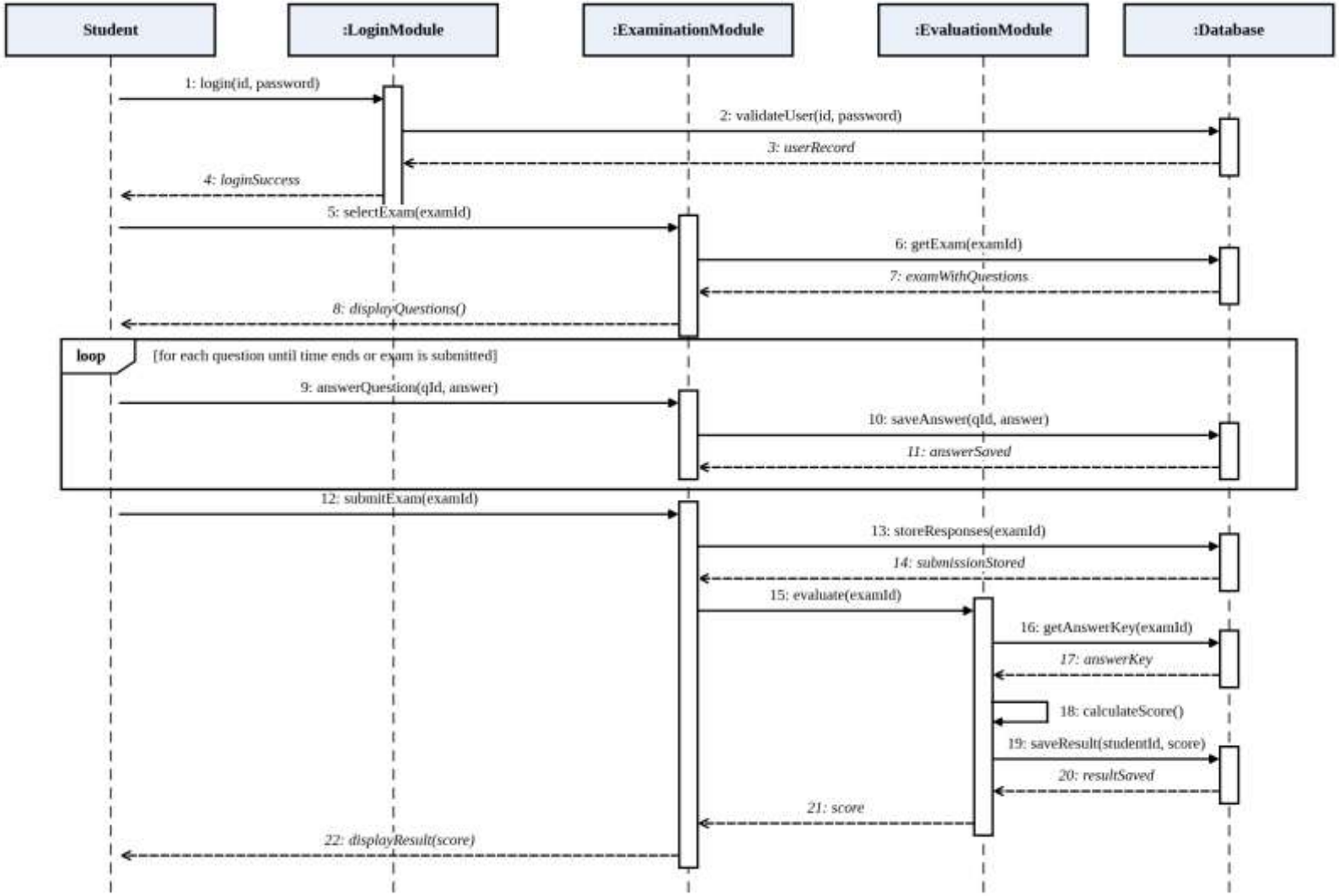


Figure 3: Sequence Diagram of the Online Examination System

4.5 Accuracy of the Solution

Check	Result
All four named participants and the added LoginModule have lifelines.	Satisfied
All seven missing exchanges (M1 to M7) appear in the message flow.	Satisfied
Messages are numbered 1 to 22 in strict top-to-bottom time order.	Satisfied
Every synchronous call has a matching return, and each activation bar starts at a call and ends at its reply.	Satisfied
The loop fragment covers only the repeated answering messages (9 to 11).	Satisfied

4.6 Interpretation and Clarity of the Solution

- **The examination module is the hub.** The student talks only to it (after login), and it uses the database and the evaluation module, so the student never touches stored data directly.
- **Saving each answer protects the student.** If the connection fails during the exam, the saved answers are not lost.
- **Evaluation is separate.** The evaluation module can be changed, for example for different marking rules, without changing the examination module.
- **Limitations.** Timers, automatic submission when time ends and handling of failed logins are not shown and can be added as further fragments.

5. Problem 4: Activity Diagram (Loan Approval System)

5.1 Understanding of the Problem Context

A Loan Approval System lets customers submit applications, upload supporting documents, go through eligibility verification, receive approval or rejection, and complete loan disbursement. The problem states that the workflow lacks proper decision points and parallel activities. A workflow with only a straight sequence of steps has three weaknesses:

- W1: Incomplete applications travel through the whole process before the problem is found.
- W2: Independent checks (documents, credit score, income) are done one after another, which makes the customer wait longer than necessary.
- W3: There is no clear approval or rejection branch, and the actions after approval are not organised.

5.2 Application of Concepts

An **activity diagram** is applied because the problem is about the flow of a business process (Booch, Rumbaugh, & Jacobson, 2005). Actions are shown as rounded boxes, a **decision node** (diamond) splits the flow by a condition written in square brackets, a **fork** bar starts activities that run in parallel and a **join** bar waits until all of them finish. A filled circle is the start and a circle with a filled centre is an end.

5.3 Problem-Solving Approach

Step 1: Identify the workflow activities.

ID	Activity	Performed By	Description
A1	Submit Loan Application	Customer	Fills in and submits the application form.
A2	Upload Supporting Documents	Customer	Uploads identity, address and income proofs.
A3	Request Missing Documents	System	Asks the customer to resubmit what is missing.
A4	Verify Documents	Loan Officer	Checks that the documents are genuine.
A5	Check Credit Score	System	Retrieves the customer's credit score.

ID	Activity	Performed By	Description
A6	Verify Income and Employment	Loan Officer	Confirms income and employment details.
A7	Evaluate Eligibility	Loan Officer	Combines all checks and applies the lending rules.
A8	Approve Loan	Loan Officer	Approves an eligible application.
A9	Reject Application	Loan Officer	Rejects an ineligible application.
A10	Notify Customer of Rejection	System	Informs the customer of the rejection.
A11	Disburse Loan Amount	System	Transfers the loan to the customer's account.
A12	Send Approval Notification	System	Informs the customer of the approval.
A13	Update Loan Records	System	Records the loan in the bank's accounts.

Step 2: Add decision nodes.

Node	Question	Guard	Next Activity
D1	Documents complete?	[complete]	Fork 1 (parallel verification)
D1	Documents complete?	[incomplete]	A3 Request Missing Documents, then back to A2
D2	Eligible?	[eligible]	A8 Approve Loan
D2	Eligible?	[not eligible]	A9 Reject Application

Step 3: Add parallel activities.

Fork / Join	Parallel Activities	Why They Can Run in Parallel
Fork 1 / Join 1	A4 Verify Documents, A5 Check Credit Score, A6 Verify Income and Employment	Each check uses different information and none needs the result of another.
Fork 2 / Join 2	A11 Disburse Loan Amount, A12 Send Approval Notification, A13 Update Loan Records	After approval these three actions are independent of each other.

Step 4: Optimize the workflow.

- The completeness check D1 comes first, so incomplete applications are sent back at once (solves W1).
- The three verifications run in parallel, so the waiting time is that of the longest check and not the sum of all three (solves W2).
- The decision D2 gives two clear outcomes, each ending in its own final node, and the post-approval actions run in parallel (solves W3).

5.4 Optimized Activity Diagram

The optimized Activity Diagram is given as Figure 4 on the next page.

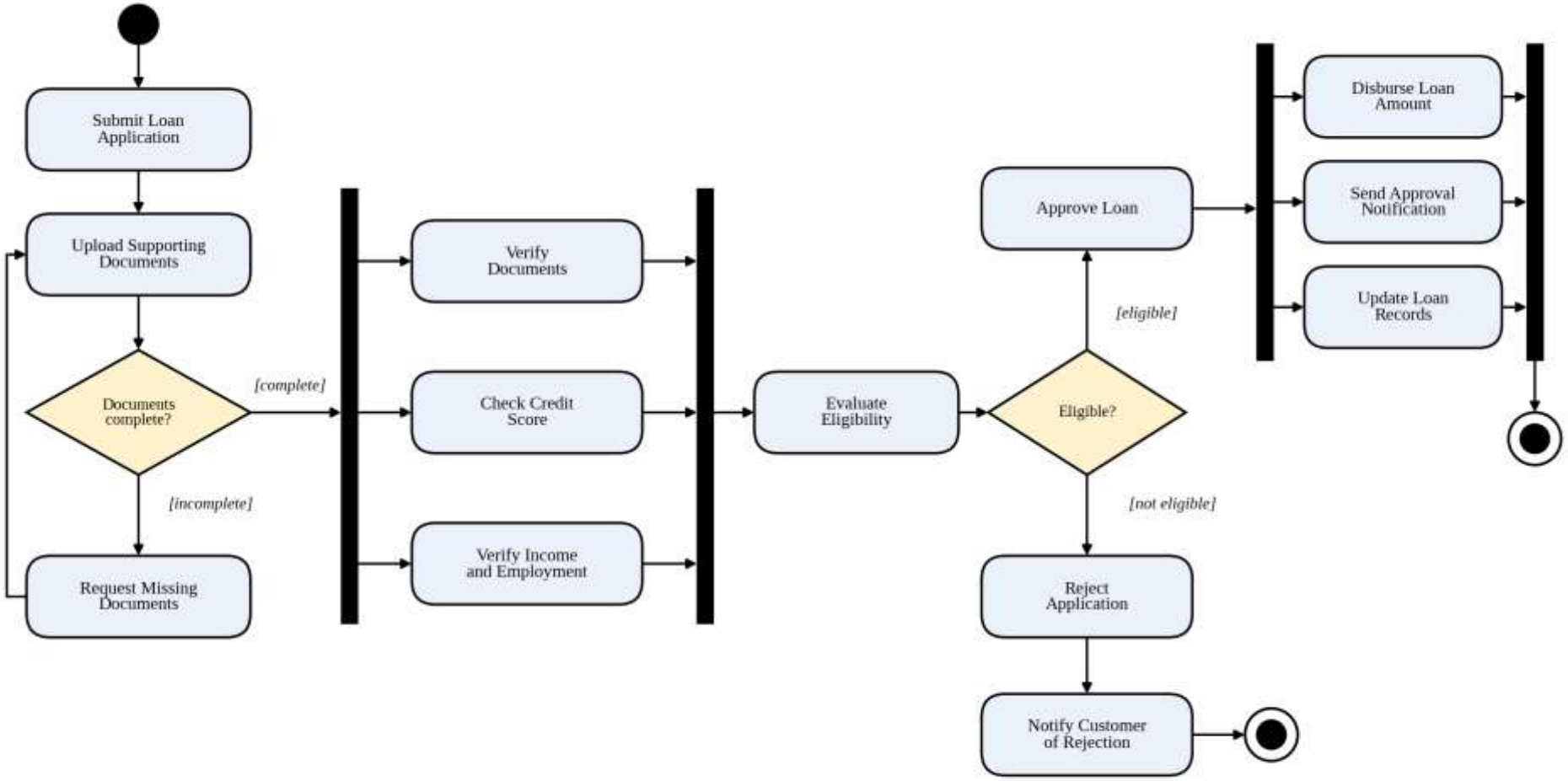


Figure 4: Activity Diagram of the Loan Approval System

5.5 Accuracy of the Solution

Check	Result
The diagram has one initial node and two final nodes (rejection end and disbursement end).	Satisfied
Every decision node has a guard on each outgoing flow, and the guards do not overlap.	Satisfied
Each fork has a matching join, and all branches of a fork reach its join.	Satisfied
Every activity A1 to A13 appears in the diagram and can be reached from the start.	Satisfied
The loop from A3 back to A2 allows resubmission without leaving the process.	Satisfied

5.6 Interpretation and Clarity of the Solution

- **Faster decisions.** Parallel verification shortens the time to decision without removing any check.
- **Less wasted effort.** Officers only start verification for complete applications.
- **Both outcomes are handled.** A rejected customer is always notified, and an approved customer always receives disbursement, a notification and an updated record.
- **Limitations.** Customer acceptance of the loan offer, repeated rejection loops and manual review of borderline cases are not shown and can be added as more decision nodes.

6. Problem 5: Component Diagram (Smart E-Commerce Platform)

6.1 Understanding of the Problem Context

The Smart E-Commerce Platform has seven modules: User Management, Product Catalog, Shopping Cart, Order Management, Payment Gateway, Inventory Management and Notification Service. The problem states that the architecture does not define the interfaces and dependencies among the modules. A user-facing Web / Mobile Client was added as an eighth component, because something has to call the platform's services.

Without defined interfaces, modules would depend directly on each other's internals. Changing one module (for example the payment provider) could then break several others. The aim is to define one clear interface for each service and to show who provides it and who requires it.

6.2 Application of Concepts

A **component diagram** is applied because the problem is about the architecture of software modules (Clements et al., 2010). A **component** is a replaceable module. A **provided interface** (a circle, or "lollipop") lists the services a component offers. A **required interface** (shown here as a dashed dependency arrow to the circle) shows a service a component needs from another. Components that only communicate through interfaces are loosely coupled.

6.3 Problem-Solving Approach

Step 1: Identify the components, with their provided and required interfaces.

Component	Responsibility	Provided Interface	Required Interfaces
Web / Mobile Client	Screens used by customers.	None	IUserAuth, IProductInfo, ICartService, IOrderService
User Management	Registration, login and profiles.	IUserAuth	INotification
Product Catalog	Product data and search.	IProductInfo	IStockCheck
Shopping Cart	Items chosen by the customer.	ICartService	IProductInfo

Component	Responsibility	Provided Interface	Required Interfaces
Order Management	Creates and tracks orders.	IOrderService	ICartService, IUserAuth, IStockCheck, IPaymentService, INotification
Payment Gateway	Authorizes payments.	IPaymentService	None
Inventory Management	Stock levels and availability.	IStockCheck	None
Notification Service	Sends email and SMS messages.	INotification	None

Step 2: Define the dependencies.

Consumer	Required Interface	Provider	Purpose
Web / Mobile Client	IUserAuth	User Management	Log in and manage the profile.
Web / Mobile Client	IProductInfo	Product Catalog	Browse and search products.
Web / Mobile Client	ICartService	Shopping Cart	Add and remove cart items.
Web / Mobile Client	IOrderService	Order Management	Place and track orders.
Shopping Cart	IProductInfo	Product Catalog	Get product price and details.
Order Management	ICartService	Shopping Cart	Read the items to be ordered.
Order Management	IUserAuth	User Management	Identify the customer.
Order Management	IStockCheck	Inventory Management	Confirm and reduce stock.
Order Management	IPaymentService	Payment Gateway	Take payment for the order.
Order Management	INotification	Notification Service	Send order messages.

Consumer	Required Interface	Provider	Purpose
Product Catalog	IStockCheck	Inventory Management	Show availability.
User Management	INotification	Notification Service	Send registration messages.

6.4 Comprehensive Component Diagram

The Component Diagram is given as Figure 5 on the next page.

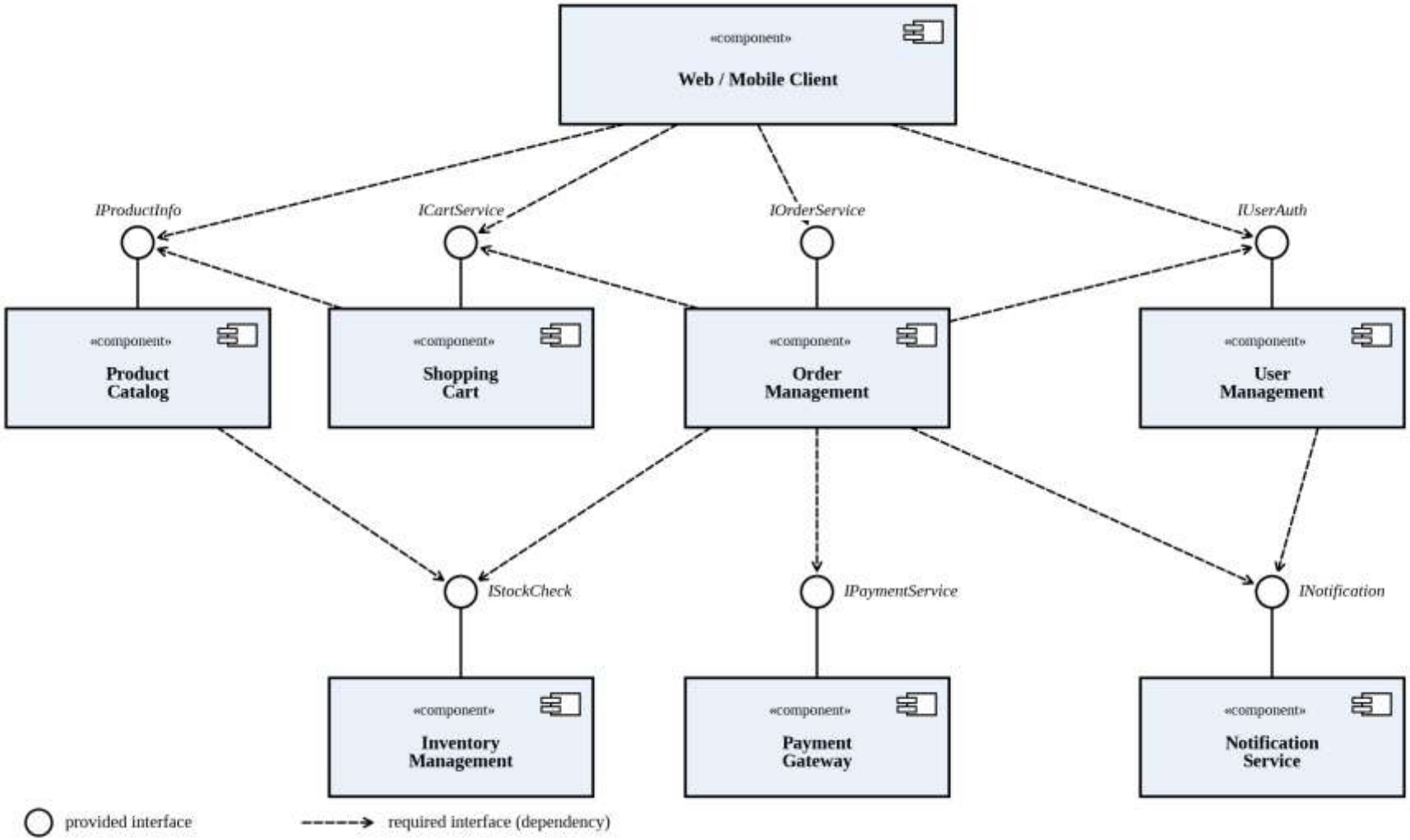


Figure 5: Component Diagram of the Smart E-Commerce Platform

6.5 Accuracy of the Solution

Check	Result
All seven named modules are present as components, plus the added client.	Satisfied
Every interface has exactly one provider and at least one consumer.	Satisfied
Every dependency in the table is drawn from the consumer to the provided interface.	Satisfied
There are no circular dependencies: Inventory, Payment and Notification depend on nothing, so the architecture can be built bottom-up.	Satisfied
Components connect only through interfaces, not directly to each other's internals.	Satisfied

6.6 Interpretation and Clarity of the Solution

- **Order Management is the orchestrator.** It is the only component that needs five other services, so the order process is controlled in one place.
- **Loose coupling.** Because the Payment Gateway is reached only through IPaymentService, a different payment provider can be used without changing Order Management.
- **Layers are visible.** The client is at the top, business components are in the middle, and the supporting services are at the bottom.
- **Limitations.** The diagram does not show how calls are made (for example direct calls or message queues) or how the services are deployed; the deployment diagram in Problem 6 shows that kind of detail.

7. Problem 6: Deployment Diagram (Smart Agriculture Monitoring System)

7.1 Understanding of the Problem Context

The Smart Agriculture Monitoring System uses IoT soil sensors, weather stations, irrigation controllers, cloud servers, mobile applications and farmer dashboards to monitor crop health and automate irrigation. The problem states that the deployment architecture does not properly show the hardware nodes and the communication links.

The system works in two directions. **Upward**, soil and weather readings travel from the field to the cloud, where they are stored and analysed. **Downward**, when the analysis finds that soil moisture is below the set level, an irrigation command travels from the cloud back to the controller in the field. Farmers see the results on a mobile app or a dashboard. An **IoT Gateway** was added as a node, because low-power field devices usually reach the cloud through a gateway.

7.2 Application of Concepts

A **deployment diagram** is applied because the problem is about the physical architecture (Clements et al., 2010; Object Management Group, 2017). A **node** is a physical device or execution environment, drawn as a 3D box. An **artifact** is a file that runs on a node (such as firmware or a service package). A **communication path** is a line between nodes, labelled with the protocol it uses.

7.3 Problem-Solving Approach

Step 1: Identify the hardware nodes.

Node	Type	Role
Soil Sensor Node	Device	Measures soil moisture, temperature and humidity in the field.
Weather Station	Device	Measures local weather such as rainfall and air temperature.
Irrigation Controller	Device	Opens and closes the irrigation valves on command.
IoT Gateway	Device (added)	Collects field data and connects the field devices to the cloud.

Node	Type	Role
Cloud Server	Execution environment	Stores and analyses data, decides irrigation and serves the apps.
Farmer Mobile Device	Device	Runs the farmer's mobile application.
Farmer PC / Dashboard	Device	Shows the farmer dashboard in a web browser.

Step 2: Specify the software artifacts deployed on each node.

Node	Artifact	Purpose
Soil Sensor Node	soil-sensor-firmware.bin	Reads the sensors and sends the readings.
Weather Station	weather-firmware.bin	Reads weather data and sends it.
Irrigation Controller	irrigation-control.bin	Receives open/close commands and drives the valves.
IoT Gateway	gateway-agent.jar	Collects field messages and forwards them to the cloud.
Cloud Server	ingestion-service.war	Receives and validates incoming sensor data.
Cloud Server	analytics-engine.jar	Analyses crop and soil conditions.
Cloud Server	control-service.jar	Sends an irrigation command when moisture is below the threshold.
Cloud Server	rest-api.war	Serves data to the mobile app and dashboard.
Cloud Server	monitoring-db.sql	Stores readings, thresholds and history.
Farmer Mobile Device	farmer-app.apk	Mobile application for monitoring.
Farmer PC / Dashboard	dashboard-ui.js	Dashboard pages run in a browser.

Step 3: Specify the communication links and protocols.

Between	Protocol	Purpose
Soil Sensor Node and IoT Gateway	LoRa	Long-range, low-power link for periodic readings.

Between	Protocol	Purpose
Weather Station and IoT Gateway	LoRa	Long-range, low-power link for weather data.
Irrigation Controller and IoT Gateway	Zigbee	Short-range link for commands and valve status.
IoT Gateway and Cloud Server	MQTT over TLS (4G / Wi-Fi)	Secure, lightweight messaging in both directions.
Farmer Mobile Device and Cloud Server	HTTPS (REST)	Secure access to data on a phone.
Farmer PC / Dashboard and Cloud Server	HTTPS (REST)	Secure access to the dashboard.

7.4 Detailed Deployment Diagram

The Deployment Diagram is given as Figure 6 on the next page.

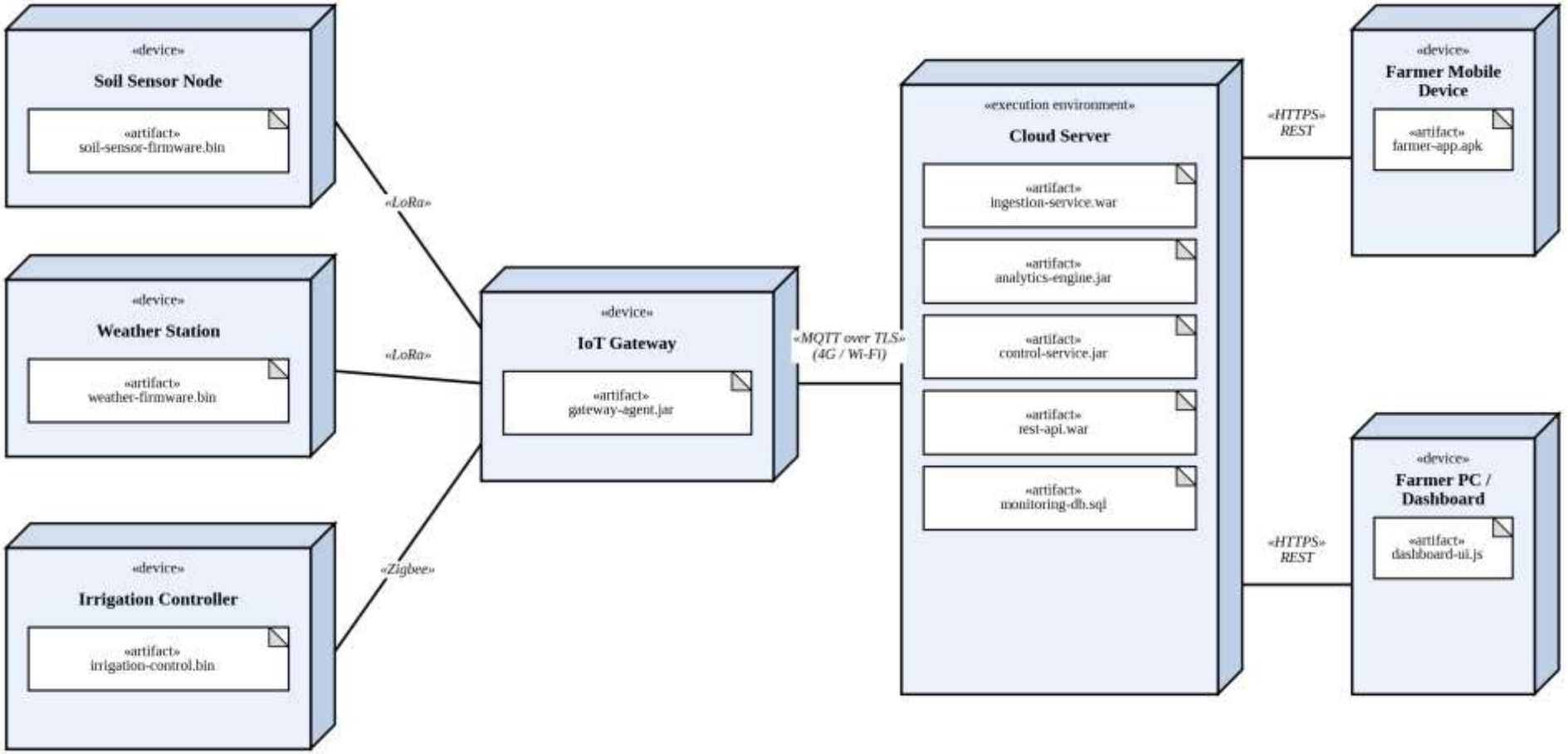


Figure 6: Deployment Diagram of the Smart Agriculture Monitoring System

7.5 Accuracy of the Solution

Check	Result
All six elements named in the problem (soil sensors, weather stations, irrigation controllers, cloud servers, mobile application and farmer dashboard) appear as nodes.	Satisfied
Every node shows its stereotype («device» or «execution environment»).	Satisfied
Every artifact is placed on the node where it runs.	Satisfied
Every communication path is labelled with a protocol.	Satisfied
No node is isolated, and every field device reaches the cloud through the gateway.	Satisfied

7.6 Interpretation and Clarity of the Solution

- **The gateway reduces load.** Field devices need only a short link to the gateway, and one secure connection carries all their data to the cloud.
- **Protocols suit the job.** LoRa suits low-power sensors spread over a large field, MQTT suits small frequent messages, and HTTPS protects what farmers see on their devices.
- **Irrigation is automated end to end.** A low moisture reading goes up to the control service, which sends a command back through the gateway to the irrigation controller.
- **Limitations.** The diagram does not show what happens during a network outage. The gateway could keep readings and run simple local rules until the connection returns.

8. Conclusion

All six problems were analysed and solved with the UML diagram that suits each one. The table summarises the result.

Problem	Diagram	Main Outcome
1	Use Case	4 actors and 16 use cases; 6 gaps closed using include, extend and generalization.
2	Class	8 classes with attributes, methods, 4 relationship types and multiplicities.
3	Sequence	5 objects and 22 messages, with 7 missing exchanges added, activation bars and a loop.
4	Activity	13 activities, 2 decision nodes, 2 parallel stages and 2 end points.
5	Component	8 components, 8 provided interfaces and 12 dependencies with no cycles.
6	Deployment	7 nodes, 11 artifacts and 6 communication paths with protocols.

Together the diagrams describe one design process: requirements (use case), structure (class and component), behaviour (sequence and activity) and physical architecture (deployment). Each solution identifies what was missing from the original description, adds it, and checks the result against the UML rules. This meets Course Outcome CO2, which is to design UML diagrams that represent system requirements and architecture.

9. Alignment with the Assessment Rubric

Rubric Criterion	Weightage	Where it is Addressed
Understanding of Problem Context	20	Sections 2.1 to 7.1 list the requirements and the gaps or weaknesses found.
Application of Concepts	30	Sections 2.2 to 7.2 justify the diagram type, and the diagrams use the correct UML notation and relationships.
Problem-Solving Approach	20	Sections 2.3 to 7.3 solve each problem step by step in tables before the diagram is drawn.
Accuracy of Solution	20	Sections 2.5 to 7.5 check each diagram against the requirements and notation rules.
Clarity	10	Landscape diagrams with consistent notation, numbered steps and an interpretation in Sections 2.6 to 7.6.

References

Booch, G., Rumbaugh, J., & Jacobson, I. (2005). *The Unified Modeling Language user guide* (2nd ed.). Addison-Wesley.

Clements, P., Bachmann, F., Bass, L., Garlan, D., Ivers, J., Little, R., Merson, P., Nord, R., & Stafford, J. (2010). *Documenting software architectures: Views and beyond* (2nd ed.). Addison-Wesley.

Cockburn, A. (2001). *Writing effective use cases*. Addison-Wesley.

Fowler, M. (2003). *UML distilled: A brief guide to the standard object modeling language* (3rd ed.). Addison-Wesley.

Jacobson, I., Christerson, M., Jonsson, P., & Övergaard, G. (1992). *Object-oriented software engineering: A use case driven approach*. Addison-Wesley.

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. https://www.omg.org/spec/UML/2.5.1